

在软件工厂中安全地使用 AI

从生成代码到生产级智能体的安全导论

Randall Degges (Snyk)



会议来源：AI Engineer World's Fair 2026

主题：安全专题导入

视频入口：[YouTube 原视频](#)

有效内容范围：主持人背景 01:32:13–01:33:27; Randall Degges 发言 01:33:34–01:37:17

核心问题：当 AI 加速软件交付时，代码、生产行动与外部可用性为何都进入安全讨论？

本文只覆盖上述安全专题导入的语义范围；原视频 01:37:42 后已进入另一场演讲，未纳入本文。

目录

1 安全为何成为软件交付的前提	2
1.1 读者应先抓住的判断	2
1.2 主持人先界定安全讨论的范围	2
1.3 Randall 从交付机会引出规模化问题	2
1.4 本章小结	3
2 第一层障碍：生成环节不免除安全责任	3
2.1 读者应先抓住的判断	3
2.2 把共同风险放回代码工件层	3
2.3 速度提升后，责任边界仍在	4
2.4 本章小结	4
3 第二层障碍：生产自主智能体的行动边界	5
3.1 学习主张：生产部署把安全问题推进到可执行行动的范围与后果	5
3.2 用四个提问元素看清行动边界	6
3.3 本章小结	6
4 第三层约束：模型可用性、外部环境与安全语境	6
4.1 学习主张：访问条件也会改变团队使用 AI 的工程处境	6
4.2 把“访问受限”与“安全语境”分开表述	7
4.3 它为何是独立的一层	7
4.4 本章小结	7
5 综合结论：以默认安全支撑无惧的 AI 使用	7
5.1 学习主张：目标是在使用 AI 时把安全作为默认工作条件	7
5.2 安全专题分会场的邀请：把问题带入持续讨论	8
5.3 本章小结	8

1 安全为何成为软件交付的前提

1.1 读者应先抓住的判断

软件工厂（software factories）把更快的交付和更多由智能体参与的协作带到工程实践的前景。对读者而言，本章的首要判断是：AI 安全（AI security）进入这一语境的原因，在于交付过程中的依赖、身份和访问范围都开始与智能体行动相连。安全由此成为软件交付中的前提条件，团队需要在交付过程中处理它。

本章核心判断

主持人给出的会场框定，与 Randall 随后的个人动机，共同指出了同一件事：AI 可以提升软件构建与交付的速度，速度需要在可理解的依赖关系、身份边界和访问范围内发挥作用。安全把交付速度与可理解的工程边界连接起来。

1.2 主持人先界定安全讨论的范围

在 Randall 上台之前，主持人 Ally How 先把安全轨道放进软件工厂的语境。她以“安全进入开发与交付闭环（security is in the loop）”描述这一前提：当软件交付越来越依赖智能体参与时，安全不能被留在流程之外。她接着把行业变化概括为从传统应用安全（traditional application security）走向智能体安全（agentic security）。这一判断来自主持人的会场框定。¹

为什么供应链、身份与访问范围属于同一背景层

主持人把智能体技能（skills）与软件包并列，称它们构成不断变化的智能体供应链（agent supply chain）。当智能体代表用户或其他智能体完成工作时，相关身份与授权关系会增加，于是出现身份蔓延（identity sprawl）的压力。她进一步提出，访问应保持为按任务收敛的访问权限（access is scoped to only what's necessary for the task at hand），并要求团队治理智能体（govern agents）。

这组表达把抽象背景转成三个连续的问题：一项任务依赖哪些能力与软件包，谁以何种身份行动，该身份的访问范围为何只覆盖当前任务所需。读者沿着这条顺序理解依赖关系、身份关系和任务范围如何共同塑造安全边界。

主持人用“似乎几乎每天都会被攻破”表达对依赖风险的担忧。读者可以把这句话理解为现场的风险提醒，并将它与可量化的事故统计区分开来。²

1.3 Randall 从交付机会引出规模化问题

Randall 从自己的开发与安全经历说起。他看重这一轮技术变化带来的机会：团队有机会更快地构建更高质量的软件，并把成果交付给真实用户。对他而言，这种速度带来的愉悦感像是获得了

¹主持人背景的原始视频时间：01:32:13–01:33:27。“智能体安全”在此作为会场概括出现，未被展开为正式标准定义。

²该段未列出事件数量、攻击案例或审计证据。

额外的加速能力；随后真正需要追问的是，团队如何把这种能力带到规模化的工程场景中。³

这个过渡决定了后文的阅读方式。安全为团队持续使用交付速度提供条件，它不要求团队放弃速度。读者可以先把注意力放在“哪些工程对象需要被安全地使用”上，再分别理解代码工件、生产行动和外部可用性的不同边界。

范围需要与任务一起被看见

一项任务的结果不能只看功能是否完成。读者还需要同时看见它依赖的能力与软件包、承担动作的身份，以及该身份在当前任务中的访问范围。把这三项放在一起，才能理解主持人为何把安全放进软件工厂的前提。

1.4 本章小结

本章建立了全文的定义域：软件工厂中的安全讨论同时触及依赖关系、身份关系和访问范围。主持人以智能体安全概括这一范围改变，Randall 则以更快、更高质量的软件交付说明 AI 的吸引力，并把规模化使用的障碍留给后续展开。读者应保留这一顺序：先理解安全为何成为交付前提，再分别考察代码工件、生产行动和外部可用性带来的不同问题。

2 第一层障碍：生成环节不免除安全责任

2.1 读者应先抓住的判断

AI 生成代码 (AI-generated code) 带来代码产出速度的变化，代码仍要承担安全责任。Randall 把这一点视为工程团队已经熟悉的风险：人编写的代码会产生安全问题，模型生成的代码也可能产生安全问题。读者可将问题从“由谁写代码”转为“代码工件进入交付之前如何继续承担安全责任”。

⁴

第一层障碍的结论

代码产出者从人扩展到 AI，安全责任依然存在。无论代码由人还是由 AI 生成，都可能带来安全问题，因此交付速度不能替代对代码工件的安全关注。

2.2 把共同风险放回代码工件层

Randall 没有把模型描绘成唯一的风险来源。他先承认，使用 AI 构建软件时，生成的代码可能带有安全问题；接着又指出，人类编写代码同样会产生安全问题。这个并列关系很重要：它避免了两种相反的误读。一种误读把模型输出视为天然不可用，另一种误读则因为生成速度很高而忽略代码本身仍是需要承担后果的工程工件。

³Randall 的机会与问题引入：01:33:34–01:34:31。

⁴本章来源：Randall 的原始视频发言 01:34:31–01:35:11。

代码工件层的关注点

本章把注意力放在代码工件可能携带的安全问题，以及它进入交付过程时仍应承担的安全责任。生产环境中智能体的行动边界，以及模型或服务的外部可用性，属于不同层次的问题。这样的划分有助于读者避免用同一种解释覆盖所有 AI 安全问题。

从这一比较可以得出的学习结论是：团队应把安全理解为开发生命周期内建安全（security as part of your development life cycle）。这里的“内建”指安全属于现代工程工作的一部分，团队在功能形成与交付的过程中持续承担这份责任。

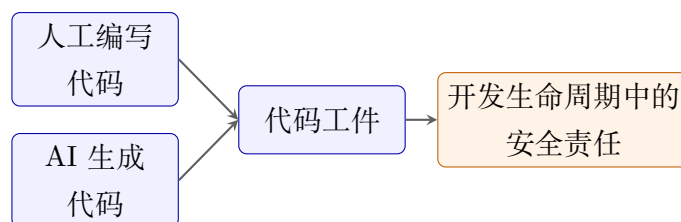


图 1：教学重绘：人工编写代码与 AI 生成代码都进入同一开发生命周期中的安全责任。该图根据 Randall 在 01:34:31–01:35:11 的对比绘制，不对应源视频中的系统架构。

2.3 速度提升后，责任边界仍在

AI 的价值在于加速构建与交付。第一层障碍提醒读者：速度改变的是产出节奏，代码工件可能产生安全问题这一边界继续存在。对学习者而言，“更快”与“仍需负责”需要同时成立：更快地产出代码，代码安全责任仍然保留。

速度提升后，责任仍然存在

把 AI 生成代码视为天然不可用，会忽略 Randall 的关键比较：人类编写代码同样会产生安全问题。把熟悉的风险视为已经解决，也会削弱开发生命周期中的安全责任。更稳妥的判断是：风险早已存在，责任始终存在。

2.4 本章小结

第一层障碍的对象是代码工件。Randall 的核心比较表明，人类编写代码与 AI 生成代码都可能产生安全问题；因此，安全仍应留在开发生命周期中。这个结论为后文建立了清晰的界限：代码层讨论产物的安全责任，后续的行动层则讨论生产环境中可执行行为的范围与后果。

3 第二层障碍：生产自主智能体的行动边界

3.1 学习主张：生产部署把安全问题推进到可执行行动的范围与后果

AI 生成代码的风险首先落在代码工件上；Randall 随后指出，把自主智能体（autonomous agents）部署到生产环境带来另一层更难的问题。团队需要判断：在部署之后，是否还能安心地让这类系统运行。Randall 用“脱离预期”来概括担忧：智能体可能做出不该做的动作，并可能伤害业务或用户（01:35:09–01:35:40）。

这里的学习重点是分清对象。代码层讨论交付出的代码是否带有安全问题；行动层讨论生产中的系统会执行什么、其行动是否越出任务边界，以及后果会落到谁身上。Randall 将后者称为更难的问题，因为系统已经处在能够行动的生产语境中。结论因此落在部署前的边界判断：团队需要知道什么行动与影响仍属于可接受范围。



图 2: Randall Degges 在 AI Engineer World's Fair 舞台上讨论生产环境中自主智能体的安全边界；镜头记录讲者与会议标识构成的现场语境。⁶

本章界限：代码工件与生产行动是两类不同的安全对象

代码层关注产物是否仍承担开发生命周期中的安全责任；行动层关注生产中的可执行行为是否仍处于被委托的任务范围内，以及越界时可能影响业务或用户。两层问题相互关联，任何一层都不能替代另一层。

⁶视频画面时间区间：01:35:20–01:35:32。

3.2 用四个提问元素看清行动边界

主持人 Ally How 在更早的铺垫中指出，智能体越来越多地代表用户或其他智能体工作，由此带来身份蔓延；她还提出**治理智能体**，并要求访问范围只保留当前任务所需的部分。这一主持人框定为 Randall 的“越界行动”担忧补上了委托与范围的背景。由此形成四个阅读元素：任务、行动、访问范围、潜在影响。它们把行动边界拆成可逐一判断的问题。

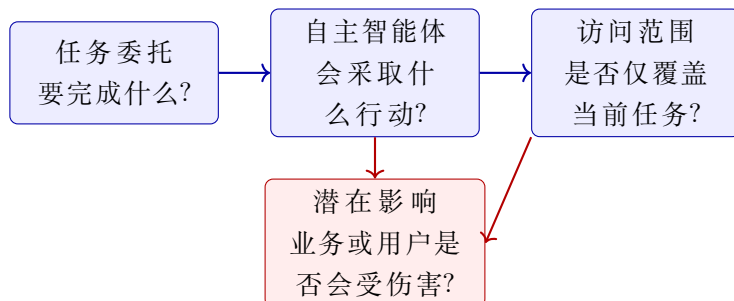


图 3: 生产自主智能体的行动边界：本笔记根据演讲风险框定所作的编辑性教学重绘，并非现场投影片。来源支撑：Randall 01:35:09–01:35:40；访问范围背景来自主持人 01:32:39–01:32:57。

先确定风险对象，再判断行动边界

图 3 中的四个元素把问题落在任务、行动、访问范围和潜在影响上。它们帮助读者区分“代码工件”与“生产行动”两类安全对象，并为后续的系统判断保留清晰的讨论起点。

3.3 本章小结

生产自主智能体使安全讨论从“代码是否有问题”延展到“系统在生产中会做什么”。任务委托、可执行行动、访问范围和业务 / 用户影响共同组成行动边界；读者据此先定位风险对象，再结合自身情境形成判断。

4 第三层约束：模型可用性、外部环境与安全语境

4.1 学习主张：访问条件也会改变团队使用 AI 的工程处境

在讨论完生产自主智能体后，Randall 把第三层障碍描述为“几乎是地缘政治”的问题（01:35:35–01:36:10）。这段发言把模型或服务能否被使用放入团队创新与交付的外部条件中。这个维度与代码工件、生产行动并列：前两层解释系统内部的安全对象，外部可用性解释团队能否取得所需能力。

他用两个现场例子让这一点变得具体：听众中有人对 Fable 的访问被撤回感到困扰，也有人对暂时无法使用新的 OpenAI GPT 5.6 模型感到不便。这两个名称标记的是 Randall 当时谈到的访问情形；它们把读者的注意力带向“团队是否能取得所需能力”这一工程问题。⁷

⁷Fable 与 OpenAI GPT 5.6 来自 Randall 在 01:35:35–01:36:10 的现场举例；来源片段未说明访问变化的成因。

外部可用性决定一部分使用条件

外部可用性进入 AI 安全语境后，团队会把模型或服务的访问状态视为工程处境的一部分。这个判断建立在“访问条件影响使用条件”上；它把讨论从单个模型的表现延展到团队持续取得能力的前提。

4.2 把“访问受限”与“安全语境”分开表述

将可用性纳入讨论并不自动回答“为什么会受限”。读者可以保留两个层次：第一层是 Randall 提到 Fable 与 OpenAI GPT 5.6 的访问情形；第二层是他的概念性关联，即这种限制会影响团队持续使用 AI 的条件。由访问情形可以建立使用条件受到影响的结论，具体成因仍需独立证据。

从例子到结论的因果边界

Fable 和 OpenAI GPT 5.6 指向 Randall 的现场举例。它们支持“访问条件会影响使用处境”的判断；单凭这两个例子，无法确定具体的外部事件或任何一方的行为。下一章将把演讲者提出的目标状态单独收束。

4.3 它为何是独立的一层

代码层的核心问题是交付物仍承担安全责任；行动层的核心问题是生产中的自主智能体可能越出任务边界；外部可用性层则询问团队是否处在能够持续取得所需模型或服务的条件下。把三层拆开，读者才能避免用“代码检查”替代行动边界，也避免用“某个模型能否访问”解释全部安全问题。

4.4 本章小结

Randall 将模型和服务的访问条件视为影响 AI 使用的第三层约束。Fable 与 OpenAI GPT 5.6 提供了当时的现场例子；外部可用性因此成为需要被识别的工程处境，具体成因需要独立证据才能判断。

5 综合结论：以默认安全支撑无惧的 AI 使用

5.1 学习主张：目标是在使用 AI 时把安全作为默认工作条件

Randall 将仍待解决的核心问题概括为：团队如何无惧地使用 AI，并让**默认安全（secure by default）**成为使用时的工作条件（01:36:07–01:36:26）。这是一种目标状态，要求团队在采用 AI 的速度与能力时始终保留安全考量。它建立的是使用方向，具体产品或系统仍需由相应证据判断。

前三层障碍为这个目标提供了清晰的阅读顺序：AI 生成代码仍承担开发生命周期中的安全责任；生产自主智能体带来可执行行动的范围与后果；模型或服务的外部可用性会改变团队的使用条件。这个顺序把“安全”从单一代码检查扩展为一套层次化判断，仍然保持每一层的边界。

全文可带走的三层安全判断

1. **代码工件：**AI 生成代码是否仍被放在开发生命周期中的安全责任内？
2. **生产行动：**自主智能体的任务与按任务收敛访问范围是否清楚，潜在影响是否被认真对待？
3. **外部可用性：**团队是否把模型或服务的访问条件当作工程约束记录下来？

这三问建立了由代码、行动到外部可用性的判断顺序；每个具体系统仍需结合自身证据回答这些问题。

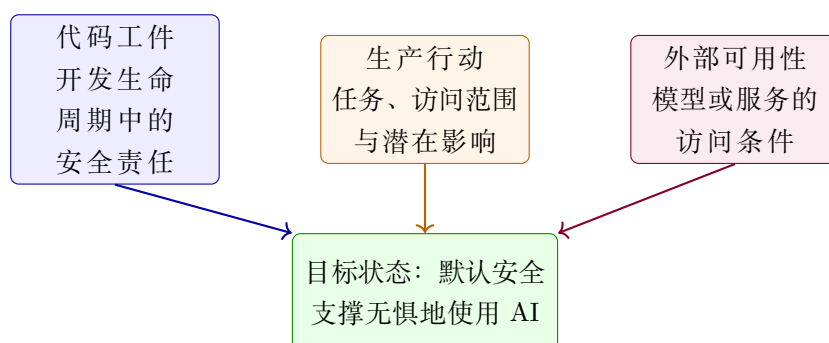


图 4：从三层障碍到“默认安全”：本笔记对本场演讲的编辑性教学综合，并非现场投影片。代码、行动与可用性分别由 01:34:33–01:35:11、01:35:09–01:35:40、01:35:35–01:36:10 支撑；目标表述来自 01:36:07–01:36:26。

5.2 安全专题分会场的邀请：把问题带入持续讨论

接着，Randall 宣布 World’ s Fair 的首个**安全专题分会场 (security track)**，说明该分会场将在主题演讲后继续讨论这些问题（01:36:24–01:37:05）。这一邀请把三层判断带入后续交流；录制时的楼层、房间和参与者名单则构成这次活动的现场背景。

5.3 本章小结

本讲以默认安全作为方向：代码工件的安全责任仍然存在，生产自主智能体的行动边界需要被正视，外部访问条件也会影响团队使用 AI 的处境。安全专题分会场是继续讨论这些问题的现场入口。这套框架帮助读者排列问题的层次，并把每一层放回具体系统与证据中判断。